

VERIQO Pilot Mode and Deployment Readiness

This document answers Commercialization Sprint items 10 and 18-20 by name: **Pilot Mode** (item 18), **minimal multi-tenancy** (item 19), **observability** (item 20), and the **production security blocker classification** (item 10). It reports the real, current state of the codebase honestly -- every "DONE" line below names the exact code and test that backs it; every "GAP" line names the exact thing that does not exist yet, rather than being silently assumed.

Item 18 -- Pilot Mode Checklist

Capability	Status	Real backing
Tenant isolation	DONE	<code>pkg/commercial/api.Store</code> keys every case/evidence record by <code>tenantID+"/"+ID</code> (<code>caseKey/evidenceKey</code>) -- a cross-tenant lookup fails at "not found" before any ownership check runs. See item 19 below for the full detail and its adversarial tests.
Case isolation	DONE	Same mechanism as tenant isolation -- each case's evidence, decision, action, and dossier are scoped to that one <code>tenantID+"/"+caseID</code> key; nothing about one case leaks into another <code>CaseView</code> , <code>Dossier</code> , or <code>Replay</code> result.
Audit	DONE	<code>pkg/platform/audit.AuditStore</code> (hash-chained, <code>Auditor{}.VerifyChain</code> -checkable) backs both the Commercial API's own Decision/Action ledger writes (<code>Store.Ledger()</code>) and every HTTP request via <code>security.AuditMiddleware</code> (<code>veriqo/gateway/rest/server.go</code>).
RBAC	DONE	<code>security.RBACMiddleware</code> (<code>pkg/platform/security/security.go</code>) gates route prefixes by JWT role; proven for the insurance routes in <code>insurance_decide_route_test.go</code> (<code>TestInsuranceDecideRouteRejectsWrongRole</code>). The Commercial API v1 routes compose the SAME middleware stack in <code>NewServer</code> -- they are not a second, parallel auth system.
API keys	DONE	<code>security.APIKeyMiddleware</code> , composed in <code>NewServer</code> alongside JWT/RBAC.
OIDC	GAP	No OpenID Connect integration exists. <code>security.JWTMiddleware</code> verifies a locally HS256-signed token (<code>security.SignHS256</code>), not a third-party IdP's token. A pilot custom wanting SSO via their own IdP needs this built; it is not a redesign of the existing JWT layer, just a second, additional verification path.
Evidence retention	DONE (P0-C)	<code>pkg/governance/data</code> 's real retention/purge lifecycle (<code>ACTIVE -> RETENTION_ELIGIBLE -> HELD -> REDACTION_REQUIRED -> REDACTED -> PURGE_ELIGIBLE -> PURGED</code> , preserving audit integrity even after content is purged) is now wired into <code>pkg/commercial/api.Store</code> 's evidence path: <code>SubmitEvidence</code> places every item under governance the moment it is finalized (required, not best-effort), and <code>pkg/commercial/api/preservation.go</code> exposes <code>SetRetentionPolicy/PlaceLegalHold/ReleaseLegalHold/EvaluateRetention/PurgeEvidence/PreservationLedger/VerifyPreservationCh</code> . Legal hold blocks purge both structurally (no <code>HELD -> PURGE_ELIGIBLE</code> edge) and via an explicit check. Six tests in <code>preservation_test.go</code> , including cross-tenant isolation preservation state. Residual gap: these capabilities are reachable via direct <code>Go Store</code> calls only -- no <code>/v1/...</code> HTTP route exposes them yet.
Export	DONE	<code>dossier.WriteMachinePackage</code> (the Evidence Dossier v1 "Machine" form) and <code>Store.WriteDossierPackage/GET /v1/cases/{id}/dossier?format=package</code> -- a real <code>.zip</code> a customer can download and independently verify.
Usage metrics	DONE	<code>pkg/commercial/telemetry + GET /v1/metrics</code> -- see item 20 below.
Health	DONE (P0-6)	<code>GET /healthz</code> reports <code>{"status":"ok","persistence":..."}</code> as before, and is now joined by separate <code>GET /livez</code> (dependency-free -- proves only that the HTTP server, so an orchestrator never restarts a healthy process over a temporarily-unready dependency) and <code>GET /readyz</code> (checks the Registry's persistence flag plus, when configured, the Commercial Store's own <code>Store.Healthy()</code> , returning 503 when it is not fit to serve). All three are exempt from the auth stack. See <code>livez_readyz_test.go</code> .
Logging	DONE	<code>loggingMiddleware</code> (<code>veriqo/gateway/rest/server.go</code>) logs method/path/status/latency for every request, with explicit log-injection sanitization (<code>strconv.Quote</code> on request path).
Support diagnostics	GAP	No dedicated support/diagnostics endpoint (e.g. a bundle of recent logs + config + version for a support ticket) exists. <code>GET /healthz</code> and <code>GET /v1/metrics</code> are the closest real artifacts a support engineer could use today.

Item 19 -- Minimal Multi-Tenancy

What the reviewer's model names: Tenant -> Users / Cases / Evidence / Policies / Decisions / Actions / Dossiers, with the Tenant ID cryptographically/authorization-bound on every path.

What actually exists:

- Cases, Evidence, Decisions, Actions, Dossiers** are all real, tenant-scoped entities in `pkg/commercial/api.Store` -- every one of `CreateCase`, `SubmitEvidence`, `DecideCase`, `ActOnCase`, `GenerateDossier`, `Replay` takes a `tenantID` and enforces it via the key-namespacing mechanism described above.
- Users -> Tenant binding is now CLOSED (P0-C's sibling item, P0-B).** The gap this section previously named -- *"TenantID is currently a caller-supplied field (request body or query parameter), not yet cryptographically bound to the verified JWT identity"* -- no longer holds. `pkg/commercial/tenancy.Membership` is the real Subject -> Tenant authorization registry, and `effectiveTenantID` (`veriqo/gateway/rest/commercial_v1_routes.go`) is the single place every tenant-scoped route resolves which tenant a request may act as. When JWT auth is configured, the caller-supplied `tenant_id` must be one that request's *verified* subject is granted -- refused with 403 otherwise, even for a perfectly valid JWT naming a different tenant -- so the invariant `effectiveTenantID == the authenticated subject's authorized tenant` holds structurally. Leaving `Membership` nil while JWT is configured fails closed (every authenticated request refused), never silently falls back to trusting the client. When no JWT middleware is configured at all, the pre-P0-B caller-supplied behavior is retained deliberately: binding is meaningless when no verified identity exists to bind to. Proven by `commercial_v1_tenant_binding_test.go` (5 tests over real signed JWTs) and `pkg/commercial/tenancy/tenancy_test.go` (6 tests).
- Policies** remain NOT first-class tenant-scoped entities: `security.RBACMiddleware`'s `RoleTable` is still global (one role-to-path-prefix map for the whole deployment). Tenant-scoped policy is a real, named, still-open item -- not silently assumed done.

The three mandatory tests the reviewer names, and where they live:

Requirement	Test
Tenant A cannot read Tenant B	<code>TestTenantAISolationFromTenantB</code> (<code>pkg/commercial/api/store_test.go</code> , 8 sub-tests) and <code>TestCommercialV1RoutesTenantIsolationOverHTTP</code> (<code>veriqo/gateway/rest/commercial_v1_routes_test.go</code>) -- both prove a 404, not tenant B's data, when tenant A requests tenant B's case or evidence by ID.
Tenant A cannot modify Tenant B	Same <code>TestTenantAISolationFromTenantB</code> sub-tests <code>cannot_modify_via_decide</code> and <code>cannot_act_on_case</code> ; <code>TestCommercialV1RoutesTenantIsolationOverHTTP</code> proves the same over HTTP for <code>POST /v1/cases/{id}/decide</code> .

Requirement	Test
Tenant A cannot replay Tenant B authorization	<code>TestTenantAISolationFromTenantB's cannot_replay_authorization</code> sub-test.

Item 20 -- Observability

Signal	Status	Real backing
Metrics	DONE	<code>pkg/commercial/telemetry.Metrics</code> , exposed at <code>GET /v1/metrics</code> . Every one of the 9 named counters is wired to a real <code>Store</code> branch (see that package's own doc comment for exactly which): <code>evidence_ingestion_total</code> , <code>evidence_verification_failures</code> , <code>custody_chain_failures</code> , <code>decision_latency</code> (as <code>decision_latency_avg_millis + decision_count</code>), <code>authorization_denials</code> , <code>action_failures</code> , <code>ledger_commit_failures</code> , <code>replay_failures</code> , <code>external_adapter_failures</code> . <code>TestMetricsReflectRealActivity</code> (<code>store_test.go</code>) and <code>TestCommercialV1RoutesMetricsReflectRealActivity</code> (<code>commercial_v1_routes_test.go</code>) prove these move with real activity, not statically. Honest exception: <code>external_adapter_failures</code> always reads zero in this reference build -- item 11's real insurer/adjuster/P&I/AIS integrations are external pilot integrations, not built in this engagement, so nothing calls an external adapter yet for this counter to ever increment on.
Logs	DONE	<code>loggingMiddleware</code> , per-request, sanitized against log injection.
Traces	GAP	No distributed tracing (OpenTelemetry spans or equivalent) exists. A request's path through JWT -> RBAC -> APIKey -> handler -> Store is visible only as one log line today, not as a span tree.
Health	DONE (P0-6)	<code>GET /healthz</code> (combined), plus the separate <code>/livez</code> and <code>/readyz</code> probes below.
Readiness / Liveness	DONE (P0-6)	<code>GET /livez</code> is deliberately dependency-free (always 200 while the process serves); <code>GET /readyz</code> is genuinely dependency-aware, wired to <code>commercialapi.Store.Healthy()</code> -- a closed durable Store makes it report 503, proven by <code>TestReadyzReportsNotReadyOnceCommercialStoreIsClosed</code> rather than assumed.
Audit	DONE	Covered under item 18 above; the same <code>pkg/platform/audit.AuditStore</code> backs both business-decision audit and HTTP-request audit.
Alerts	GAP	No alerting integration (paging, threshold-based notification) exists. <code>GET /v1/metrics'</code> real counters are what a pilot deployment's own monitoring stack would need to scrape and alert on -- this codebase does not push alerts itself.

Item 10 -- Production Security Blocker Classification

This engagement's earlier rounds already built and honestly tracked the 8 production blockers item 10 asks for -- see `docs/governance/production-blockers.json` (the machine-readable contract) and `evidence/blockers-qualification-report.json` (the live, independently-recomputed qualification result, not a stale label). Reproduced here for this document's own completeness, in item 10's own MUST-CLOSE vs. CAN-BE-PILOT-ENVIRONMENT framing:

MUST CLOSE BEFORE PAID PILOT (per item 10's own list):

- HSM/KMS-backed production signing -- blocker `hsm_kms`
- Durable persistent ledger -- **CLOSED (P0-A)**: `NewDurableStore` (`pkg/commercial/api/durable.go`) backs every mutating Commercial API call with the real `pkg/storage/wal` write-ahead log (fsync, CRC, defect-classified recovery), reconstructing byte-identical state on restart by replaying recorded inputs through the Store's own core logic. Because that replay covers the audit ledger too, "production audit durability" is substantively closed by the same mechanism. Proven by `TestNewDurableStoreReconstructsIdenticalStateAfterRestart`, `TestDurableStoreRecoversFromATornWrite` (a real crash-mid-write, not just a clean shutdown), and a live restart test against the compiled `verigo-gateway` binary. `NewStore` remains in-memory-only and unchanged for callers that want it.
- Production identity/mTLS -- blocker `spire_mtls`
- Secure object storage + retention -- partially covered by `pkg/governance/data's` retention lifecycle (see item 18 above); no dedicated "secure object storage" blocker entry exists for the evidence bytes themselves (this codebase stores evidence metadata and hashes in `manifest.Manifest`, never the raw bytes -- object storage for the raw bytes is external to this codebase by design, and its security is a deployment-time concern, not something this repository can qualify on its own).
- Vulnerability scanning -- blocker `supply_chain_scan`
- Backup/restore -- **MECHANISM CLOSED (P0-A)**: `Store.Backup` copies the whole durable WAL (fsyncing first, so the copy includes every record a caller was told was durably written), and `RestoreStoreFromBackup` replays it through the exact same code path as ordinary crash recovery -- a backup is deliberately not a separate, untested format. `TestStoreBackupAndRestoreRoundTrip` proves the round trip; `Backup` returns `ErrNotDurable` rather than pretending for an in-memory Store. **Residual gap:** no written operator runbook and no drill against a live deployment.
- Production logging/monitoring -- item 20 above is the current state; genuinely partial (metrics/logs/audit real, traces/alerts/separate health probes are gaps).
- Security incident response procedure -- **no dedicated blocker entry**; this is an organizational/process artifact, not code, and is named here as outstanding rather than silently assumed to exist.

CAN BE PILOT ENVIRONMENT (per item 10's own list; must show `NOT_YET_PRODUCTION_QUALIFIED`, never hidden):

- Multi-region -- blocker `multi_region_dr`
- 100-node physical qualification -- blocker `scale_qualification`
- 72-hour soak at production scale -- blocker `soak_72h`
- Independent penetration test -- blocker `pentest`

Current status of all 8 tracked blockers (from `evidence/blockers-qualification-report.json`, this session's own fresh read, not a cached or stale claim): every one of `pentest`, `scale_qualification`, `multi_region_dr`, `hsm_kms`, `live_data`, `soak_72h`, `spire_mtls`, `supply_chain_scan` reads `READY_FOR_REAL_QUALIFICATION`. That status means: the MECHANISM this codebase would need for a real qualification run (fixture-mode and synthetic-mode test harnesses, failure-injection matrices, measurement capture) is built and proven; the REAL external qualification itself (an actual signed pentest vendor report, actual 100 physical/cloud nodes, actual procured HSM/KMS tenancy, actual multi-region infrastructure, an actual 72-hour run at production scale) has not happened, because it requires real external infrastructure and vendors this engagement does not have. Per item 10's own instruction, this is reported here as `NOT_YET_PRODUCTION_QUALIFIED`, not hidden or implied otherwise.

Honest Summary

Pilot Mode's tenant/case isolation, audit, RBAC, API-key auth, evidence export, and usage metrics are real and tested. Since this document was first written, four of the gaps it named have been genuinely closed and are marked as such above with their backing tests: **durable persistence and backup/restore (P0-A)**, **tenant identity binding (P0-B)**, **evidence-retention/legal-hold integration into the Commercial API path (P0-C)**, and **separate liveness/readiness probes (P0-6)** -- along with cryptographic signing (P0-D) and an independent verifier that performs real signature/key-state verification (P0-E).

Still genuinely open, named rather than silently assumed: OIDC, distributed tracing, alerting, support diagnostics, tenant-scoped policy, HTTP routes for the retention/signing capabilities, an incident response procedure, and an operational backup drill. Every one of the 8 tracked production security blockers still has a proven mechanism but no completed real-world external qualification.

For the precise, per-tier readiness verdict that replaces this document's earlier single-phrase "pilot-ready under the caveats above, not yet fully production-qualified" framing, see **docs/VERIQO_READINESS_TIER_FRAMEWORK.md**: DEMO-READY yes, DESIGN-PARTNER READY yes, PAID-PILOT READY conditionally (gating items closed; operational/documentation conditions enumerated there), NOT PRODUCTION QUALIFIED.